

Phase 2 - Databases

First a Word from Our Sponsor

For my DM project,

CONCAT to be able to do a replace for team's points

UNIQUE to get list of teams

I'm using this stuff!

Databases & SQL Day 1

Warning! This Is NOT A Databases Course!

- Simplest methods here - just enough to get you going
- Leaving out tons of things (security, proper design, etc.)
- Going to do more with databases? Take a DB course!

Database Motivation

- Simultaneous access (unlike Excel)
- Reliable storage
- Easy to get information (write one query instead of spending lots of time in Excel)
- Efficiency (don't save duplicated data, only make one update when something changes)
- Optimized for speed

Different Databases

Types (**relational**, **noSQL**, hierarchical, object-oriented, network)

- Relational: tables with links (relations)
- NoSQL: not tabular, typically for big data & real-time web applications

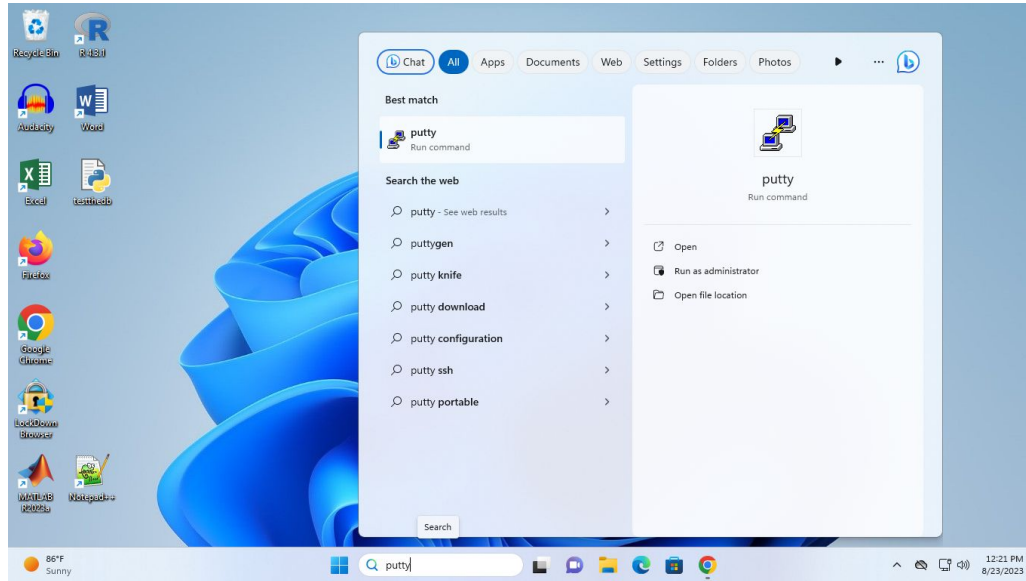
Versions: **MySQL**, Postgres, SQLServer, Access, etc.

Accessing Databases

- Command line
- Programming languages

Before We Can Access With The Command Line...

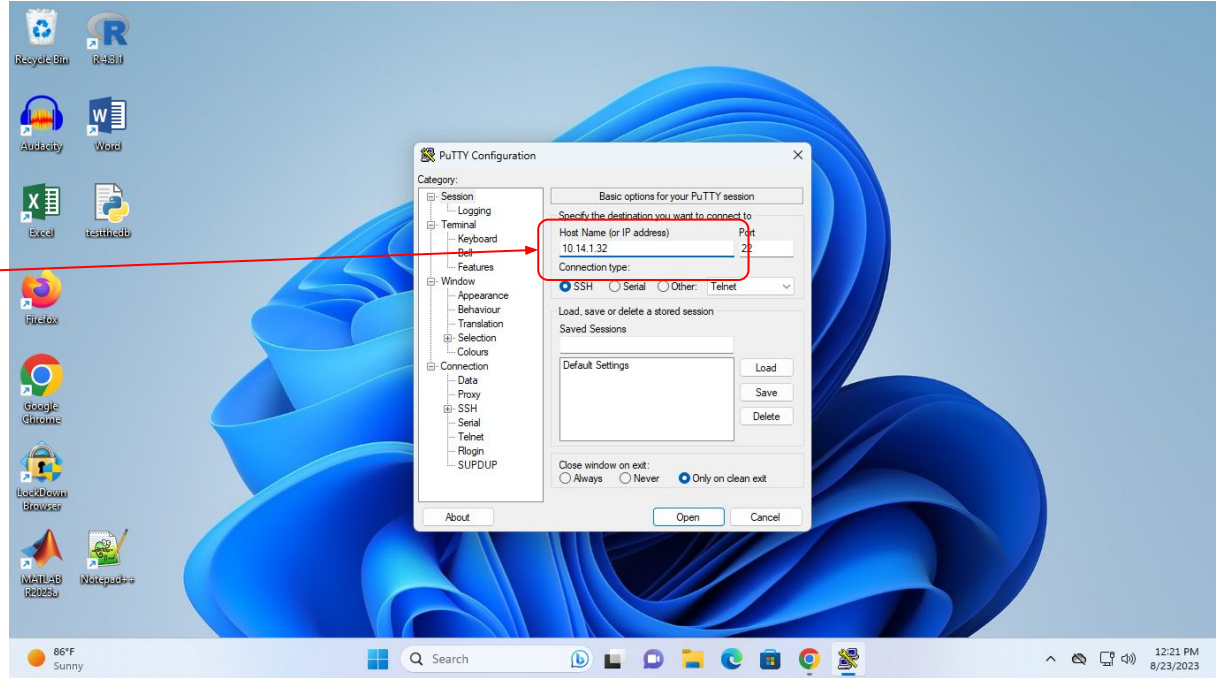
Need to connect to the computer with the DBMS - use putty



Using putty

the server for the course

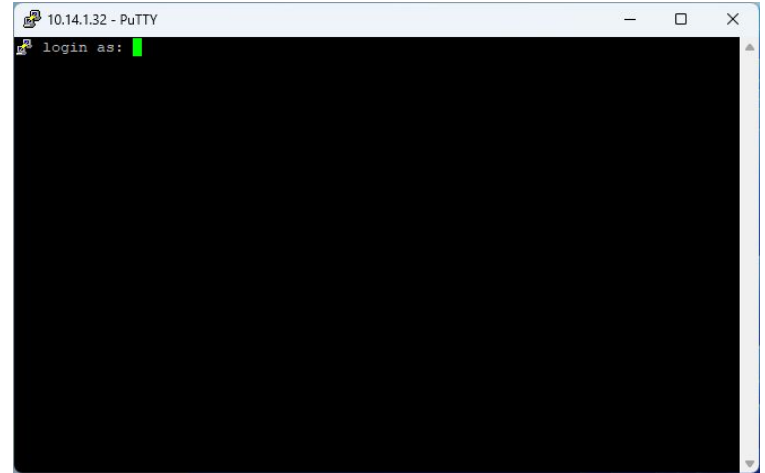
Enter `cscdata.centre.edu`
here



Before We Can Access With The Command Line...

Enter your Centre College username and hit Enter:

When you enter your password, there will be NO visual indication of you entering characters (for security)



Using MySQL With The Command Line

-- Start a session with the command line

- Typically: `mysql -u your_username -p`

-- Your username is

rudu, finn, drew, gabriel, noah, kyle, leslie, ninh, porter, hung, josh, ben, arthur, ha, pranjai

-- Enter your database password when prompted: **DSC270F@112023**

-- NOTE: You will not see any indication of characters being entered as you type!

-- Change your password now like this:

set password for '**rudu**'@'%' = '**rudu12345678**';

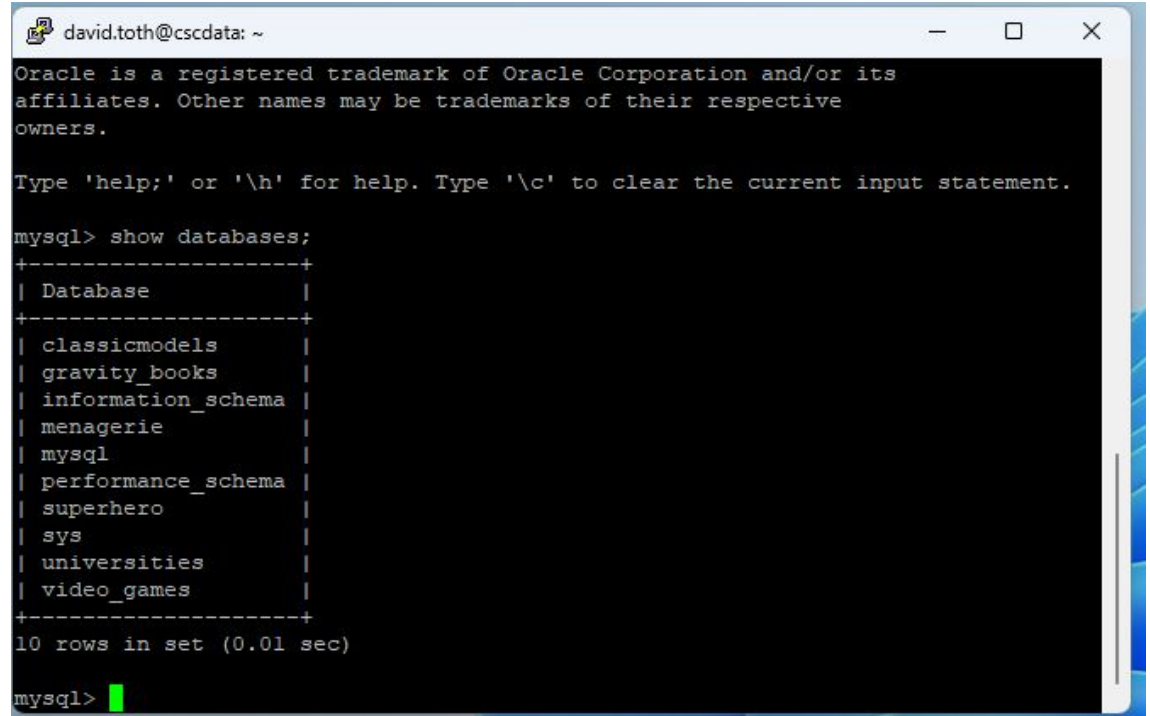
Replace the text in blue with your username and your new password.

Listing Existing Databases

-- Show the existing databases

SHOW DATABASES;

-- Note there will be some there by default

A terminal window titled 'david.toth@cscdata: ~' with standard window controls. It shows the MySQL command 'show databases;' and its output, which is a table of database names. The output includes 'classicmodels', 'gravity_books', 'information_schema', 'menagerie', 'mysql', 'performance_schema', 'superhero', 'sys', 'universities', and 'video_games'. The terminal also shows a copyright notice for Oracle at the top and a help message below it. The prompt 'mysql>' is visible at the bottom with a green cursor.

```
david.toth@cscdata: ~
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| classicmodels |
| gravity_books |
| information_schema |
| menagerie |
| mysql |
| performance_schema |
| superhero |
| sys |
| universities |
| video_games |
+-----+
10 rows in set (0.01 sec)

mysql>
```

Using Databases

-- Use the database dbname

USE dbname ;

SHOW TABLES

What tables are in the database?

SHOW TABLES;

DESCRIBE

DESCRIBE table gets you the info about a table

DESCRIBE sometable;

The fields are the data.

The type indicates what kind of data/how it's stored

Ignore the other columns for now.

Getting Data From A Database

SELECT

FROM

JOIN

WHERE

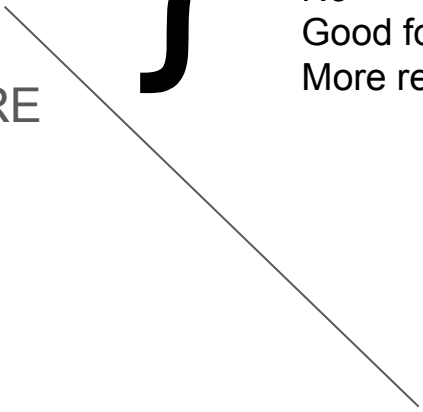


Must these all be all caps?

No

Good form

More readable code



Not today!

SELECT and FROM

Get the specified columns from a table

* means all columns

SELECT * FROM Candy;

SELECT Fat, Name FROM Candy;

```
mysql> SELECT Fat, Name FROM Candy;
+-----+-----+
| Fat | Name |
+-----+-----+
| 4   | Nerds |
| 6.7 | Twix  |
| 2   | Milky Way |
+-----+-----+
3 rows in set (0.00 sec)

mysql> █
```

Candy

Name	Calories	Fat	Sugar	Carbs
Nerds	100	4	34	12
Twix	250	6.7	56	5
Milky Way	200	2	35	9

```
mysql> SELECT * FROM Candy;
+-----+-----+-----+-----+-----+
| Name | Calories | Fat | Sugar | Carbs |
+-----+-----+-----+-----+-----+
| Nerds | 100 | 4 | 34 | 12 |
| Twix | 250 | 6.7 | 56 | 5 |
| Milky Way | 200 | 2 | 35 | 9 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> █
```

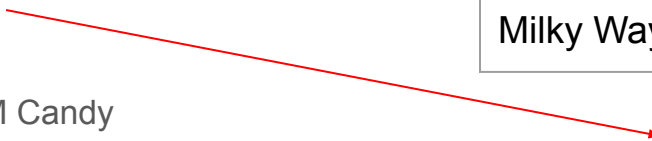
WHERE

Restricts the output

SELECT Fat, Name FROM Candy
WHERE Calories < 220;

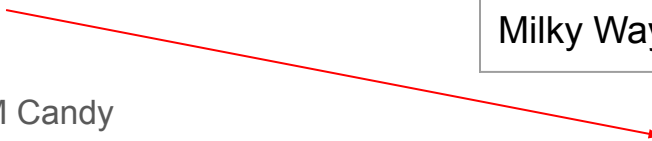
Name	Calories	Fat	Sugar	Carbs
Nerds	100	4	34	12
Twix	250	6.7	56	5
Milky Way	200	2	35	9

SELECT Fat, Name FROM Candy
WHERE Calories < 220 AND Fat > 3;



Fat	Name
4	Nerds

SELECT Fat, Name FROM Candy
WHERE Fat = 4 OR Fat = 6.7;



Fat	Name
4	Nerds
2	Milky Way



Fat	Name
4	Nerds
6.7	Twix

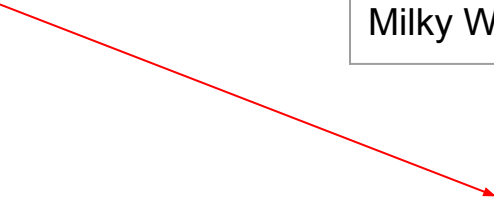
WHERE

Restricts the output

SELECT Fat, Name FROM Candy

WHERE Name = 'Nerds';

Name	Calories	Fat	Sugar	Carbs
Nerds	100	4	34	12
Twix	250	6.7	56	5
Milky Way	200	2	35	9



Fat	Name
4	Nerds

SELECT Fat, Name FROM Candy

WHERE Name = 'Nerds' OR Name = 'Twix';



Fat	Name
4	Nerds
6.7	Twix

NULL

is NULL instead of = NULL

example:

```
SELECT * FROM some_table WHERE some_col IS NULL
```

Lab Time!

Lab is linked from Moodle.

Databases & SQL Day 2

Today's Keywords & Topics

Keywords

- DISTINCT
- COUNT
- NOT
- ORDER BY
 - ASC
 - DESC
- MIN
- MAX
- SUM
- AVG
- BETWEEN

Topics

- Arithmetic
- Dates

Today's Sample Database

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> select * from players;  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| lastName | firstName | birthDate | goals | assists | yellowCards | redCards | team | position |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| Bright | Millie | 1993-01-04 | 0 | 0 | 5 | 1 | Chelsea | DF |  
| Cuthbert | Erin | 1993-05-23 | 4 | 5 | 2 | 0 | Chelsea | MF |  
| Daly | Rachel | 1991-08-21 | 12 | 3 | 7 | 1 | Aston Villa | FW |  
| Foord | Caitlin | 1994-10-15 | 5 | 7 | 2 | 0 | Arsenal | FW,MF |  
| McCabe | Katie | 1995-11-17 | 1 | 6 | 12 | 2 | Arsenal | DF,FW |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> 
```

Only Want Unique Items? Use **DISTINCT**

```
mysql> SELECT team FROM players;
+-----+
| team  |
+-----+
| Chelsea |
| Chelsea |
| Aston Villa |
| Arsenal |
| Arsenal |
+-----+
5 rows in set (0.00 sec)

mysql> █
```

```
mysql> SELECT DISTINCT team FROM players;
+-----+
| team  |
+-----+
| Chelsea |
| Aston Villa |
| Arsenal |
+-----+
3 rows in set (0.00 sec)

mysql> █
```

Want to Know How Many Things? Use COUNT

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> SELECT COUNT(team) FROM players;  
+-----+  
| COUNT(team) |  
+-----+  
|          5 |  
+-----+  
1 row in set (0.00 sec)  
  
mysql> SELECT COUNT (team) FROM players;  
ERROR 1630 (42000): FUNCTION soccer.COUNT does not exist. Check the 'Function Name Parsing and Resolution' section in the Reference Manual  
mysql> 
```

Careful about your spacing with COUNT!

Want to Know How Many Different Things?

Use **COUNT** and **DISTINCT**

```

david.toth@cscdata: ~
File Edit View Search Terminal Help
mysql> SELECT COUNT(DISTINCT(team)) FROM players;
+-----+
| COUNT(DISTINCT(team)) |
+-----+
|                      3 |
+-----+
1 row in set (0.00 sec)

mysql> 
```

Want to Negate Something? Use **NOT**

```
mysql> SELECT lastName FROM players WHERE NOT lastName = 'Cuthbert';
+-----+
| lastName |
+-----+
| Bright   |
| Daly     |
| Foord    |
| McCabe   |
+-----+
4 rows in set (0.01 sec)

mysql> 
```

Want to Sort? Use **ORDER BY** w/ **ASC** or **DESC**

```
mysql> SELECT lastName, firstName, goals FROM players ORDER BY goals;
```

lastName	firstName	goals
Bright	Millie	0
McCabe	Katie	1
Cuthbert	Erin	4
Foord	Caitlin	5
Daly	Rachel	12

5 rows in set (0.00 sec)

```
mysql> █
```

david.toth@cscdata: ~

File Edit View Search Terminal Help

```
mysql> SELECT lastName, firstName, goals FROM players;
```

lastName	firstName	goals
Bright	Millie	0
Cuthbert	Erin	4
Daly	Rachel	12
Foord	Caitlin	5
McCabe	Katie	1

5 rows in set (0.00 sec)

```
mysql> █
```

Want to Sort? Use **ORDER BY** w/ **ASC** or **DESC**

```
mysql> SELECT lastName, firstName, goals FROM players ORDER BY goals ASC;
+-----+-----+-----+
| lastName | firstName | goals |
+-----+-----+-----+
| Bright   | Millie   | 0     |
| McCabe   | Katie    | 1     |
| Cuthbert | Erin     | 4     |
| Foord    | Caitlin  | 5     |
| Daly     | Rachel   | 12    |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT lastName, firstName, goals FROM players ORDER BY goals DESC;
+-----+-----+-----+
| lastName | firstName | goals |
+-----+-----+-----+
| Daly     | Rachel   | 12    |
| Foord    | Caitlin  | 5     |
| Cuthbert | Erin     | 4     |
| McCabe   | Katie    | 1     |
| Bright   | Millie   | 0     |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> █
```

Want to Sort? Use **ORDER BY** w/ **ASC** or **DESC**

```
mysql> SELECT lastName, redCards, position FROM players ORDER BY redCards;
```

lastName	redCards	position
Cuthbert	0	MF
Foord	0	FW,MF
Bright	1	DF
Daly	1	FW
McCabe	2	DF,FW

5 rows in set (0.00 sec)

```
mysql> SELECT lastName, redCards, position FROM players ORDER BY redCards, position;
```

lastName	redCards	position
Foord	0	FW,MF
Cuthbert	0	MF
Bright	1	DF
Daly	1	FW
McCabe	2	DF,FW

5 rows in set (0.00 sec)

```
mysql> █
```

Want Summary Data? Use **MAX**, **MIN**, **SUM**, **AVG**

```
mysql> SELECT MAX(goals) FROM players;
+-----+
| MAX(goals) |
+-----+
|          12 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT MIN(goals) FROM players;
+-----+
| MIN(goals) |
+-----+
|           0 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> SELECT SUM(goals) FROM players;
+-----+
| SUM(goals) |
+-----+
|          22 |
+-----+
1 row in set (0.02 sec)
```

```
mysql> SELECT AVG(goals) FROM players;
+-----+
| AVG(goals) |
+-----+
|       4.4000 |
+-----+
1 row in set (0.00 sec)
```

Want Something in a Range? Use **BETWEEN**

```
mysql> SELECT lastName, goals FROM players WHERE goals BETWEEN 1 AND 5;
+-----+-----+
| lastName | goals |
+-----+-----+
| Cuthbert |     4 |
| Foord    |     5 |
| McCabe   |     1 |
+-----+-----+
3 rows in set (0.00 sec)

mysql> 
```

Arithmetic

```
mysql> SELECT lastName, (goals + assists) FROM players;
```

lastName	(goals + assists)
Bright	0
Cuthbert	9
Daly	15
Foord	12
McCabe	7

5 rows in set (0.01 sec)

```
mysql> █
```

```
mysql> SELECT lastName FROM players WHERE (goals + assists) > 10;
```

lastName
Daly
Foord

2 rows in set (0.00 sec)

```
mysql> █
```

```
mysql> SELECT lastName, (goals + assists) FROM players WHERE (goals + assists) > 10;
```

lastName	(goals + assists)
Daly	15
Foord	12

2 rows in set (0.00 sec)

```
mysql> █
```

Using Dates

Constructing a DATE

`DATE('yyyy-mm-dd')`

Components of a date can be retrieved using a function that takes the date

example: `SELECT DAYOFMONTH(some_date) FROM some_table;`

example: `SELECT MONTHNAME(some_date) FROM some_table;`

example: `SELECT YEAR(some_date) FROM some_table;`

example: `SELECT DAYNAME(some_date) FROM some_table;`

Can also `SELECT CURDATE();`

Times

There are analogous functions for times.

Lab Time!

Lab is linked from Moodle.

Databases & SQL Day 3

Today's Keywords & Topics

Keywords

- LIMIT
- CREATE TABLE
- LIKE
- CONCAT & CONCAT_WS
- TRIM, LTRIM, RTRIM
- AS
- GROUP BY
- HAVING
- JOIN

Topics

- Limiting Results
- Proper Ordering
- Creating Tables Dynamically
- Comparing Lists
- Evaluation Order
- Matching Strings
- Concatenation
- Removing Spaces
- Aliases (AS)
- Grouping
- Filtering Grouped Data
- Sorting Filtered Data
- ER Diagrams
- Joins

Limiting Results

LIMIT (MySQL - different for different DBMS's)

SELECT columns FROM table WHERE condition LIMIT xyz;

Proper Ordering

Clause	Specifies
SELECT	Column/s or expressions to return
FROM	Table to retrieve data from
WHERE	Row-level filter
GROUP BY	Column to group around
HAVING	Group-level filter
ORDER BY	Return sort order

LIMIT

Limit the number of results (MySQL only)

Creating Tables Dynamically

```
mysql> select * from microwaves;
+----+-----+-----+-----+
| id | maker  | model  | power |
+----+-----+-----+-----+
| 1  | Sharp  | R254SL | 800   |
| 2  | Sharp  | R33STM | 900   |
| 3  | Sanyo  | EMS3553 | 900   |
| 4  | Panasonic | NNE442 | 900   |
| 5  | Daewoo | KDR3000 | 800   |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> █
```

→

```
mysql> CREATE TABLE 800w_microwaves
      -> SELECT * FROM microwaves WHERE power = 800;
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> █
```

↓

```
mysql> SELECT * FROM 800w_microwaves;
+----+-----+-----+-----+
| id | maker  | model  | power |
+----+-----+-----+-----+
| 1  | Sharp  | R254SL | 800   |
| 5  | Daewoo | KDR3000 | 800   |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> █
```

CREATE TABLE new_table_name
SELECT columns FROM table WHERE condition;

Comparing Lists

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> select * from players;  
+-----+-----+-----+-----+-----+-----+-----+-----+  
| lastName | firstName | birthDate | goals | assists | yellowCards | redCards | team | position |  
+-----+-----+-----+-----+-----+-----+-----+-----+  
| Bright | Millie | 1993-01-04 | 0 | 0 | 5 | 1 | Chelsea | DF |  
| Cuthbert | Erin | 1993-05-23 | 4 | 5 | 2 | 0 | Chelsea | MF |  
| Daly | Rachel | 1991-08-21 | 12 | 3 | 7 | 1 | Aston Villa | FW |  
| Foord | Caitlin | 1994-10-15 | 5 | 7 | 2 | 0 | Arsenal | FW,MF |  
| McCabe | Katie | 1995-11-17 | 1 | 6 | 12 | 2 | Arsenal | DF,FW |  
+-----+-----+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT lastName FROM players WHERE team IN ('Chelsea', 'Aston Villa');  
+-----+  
| lastName |  
+-----+  
| Bright |  
| Cuthbert |  
| Daly |  
+-----+  
3 rows in set (0.01 sec)  
  
mysql> 
```

SELECT lastName FROM
players WHERE team IN
(‘Chelsea’, ‘Aston Villa’);

Evaluation Order

AND is evaluated before OR

Change order or make it explicit with parentheses ()

Matching Strings

% is wildcard

LIKE

```
mysql> SELECT firstName FROM players WHERE firstName LIKE '%in%';
```

```
+-----+  
| firstName |  
+-----+  
| Erin      |  
| Caitlin   |  
+-----+  
2 rows in set (0.00 sec)
```

```
mysql> █
```

```
mysql> SELECT firstName FROM players WHERE firstName LIKE '%in';
```

```
+-----+  
| firstName |  
+-----+  
| Erin      |  
| Caitlin   |  
+-----+  
2 rows in set (0.00 sec)
```

```
mysql> █
```

```
mysql> SELECT birthDate FROM players WHERE birthDate LIKE '1993%';
```

```
+-----+  
| birthDate |  
+-----+  
| 1993-01-04 |  
| 1993-05-23 |  
+-----+
```

```
2 rows in set (0.00 sec)
```

```
mysql> █
```

Concatenation

Varies by DBMS, MySQL has CONCAT and CONCAT_WS

```
mysql> SELECT CONCAT(firstName, lastName) FROM players;
```

```
+-----+
| CONCAT(firstName, lastName) |
+-----+
| MillieBright                |
| ErinCuthbert                |
| RachelDaly                  |
| CaitlinFoord                |
| KatieMcCabe                 |
+-----+
```

```
5 rows in set (0.00 sec)
```

```
mysql> █
```

```
mysql> SELECT CONCAT_WS(", ", lastName, firstName) FROM players;
```

```
+-----+
| CONCAT_WS(", ", lastName, firstName) |
+-----+
| Bright, Millie                      |
| Cuthbert, Erin                      |
| Daly, Rachel                        |
| Foord, Caitlin                      |
| McCabe, Katie                       |
+-----+
```

```
5 rows in set (0.00 sec)
```

```
mysql> █
```

Removing Spaces

TRIM

LTRIM

RTRIM

Aliases

```
mysql> SELECT CONCAT_WS(", ", lastName, firstName) FROM players;
```

```
+-----+  
| CONCAT_WS(", ", lastName, firstName) |  
+-----+  
| Bright, Millie |  
| Cuthbert, Erin |  
| Daly, Rachel |  
| Foord, Caitlin |  
| McCabe, Katie |
```

```
5 rows in set (0.00 s)
```

```
mysql> █
```

```
mysql> SELECT CONCAT_WS(", ", lastName, firstName) AS 'Full Name' FROM players;
```

```
+-----+  
| Full Name |  
+-----+  
| Bright, Millie |  
| Cuthbert, Erin |  
| Daly, Rachel |  
| Foord, Caitlin |  
| McCabe, Katie |
```

```
5 rows in set (0.00 sec)
```

```
mysql> █
```

Grouping

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> select * from players;  
+-----+-----+-----+-----+-----+-----+-----+-----+  
| lastName | firstName | birthDate | goals | assists | yellowCards | redCards | team | position |  
+-----+-----+-----+-----+-----+-----+-----+-----+  
| Bright | Millie | 1993-01-04 | 0 | 0 | 5 | 1 | Chelsea | DF |  
| Cuthbert | Erin | 1993-05-23 | 4 | 5 | 2 | 0 | Chelsea | MF |  
| Daly | Rachel | 1991-08-21 | 12 | 3 | 7 | 1 | Aston Villa | FW |  
| Foord | Caitlin | 1994-10-15 | 5 | 7 | 2 | 0 | Arsenal | FW,MF |  
| McCabe | Katie | 1995-11-17 | 1 | 6 | 12 | 2 | Arsenal | DF,FW |  
+-----+-----+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql>   
  
mysql> SELECT team, MAX(goals) FROM players GROUP BY team;  
+-----+-----+  
| team | MAX(goals) |  
+-----+-----+  
| Chelsea | 4 |  
| Aston Villa | 12 |  
| Arsenal | 5 |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> 
```

Do aggregate function on
each group!

Grouping

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> select * from players;  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| lastName | firstName | birthDate | goals | assists | yellowCards | redCards | team | position |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| Bright | Millie | 1993-01-04 | 0 | 0 | 5 | 1 | Chelsea | DF |  
| Cuthbert | Erin | 1993-05-23 | 4 | 5 | 2 | 0 | Chelsea | MF |  
| Daly | Rachel | 1991-08-21 | 12 | 3 | 7 | 1 | Aston Villa | FW |  
| Foord | Caitlin | 1994-10-15 | 5 | 7 | 2 | 0 | Arsenal | FW,MF |  
| McCabe | Katie | 1995-11-17 | 1 | 6 | 12 | 2 | Arsenal | DF,FW |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT team, SUM(yellowCards) FROM players GROUP BY team;  
+-----+-----+  
| team | SUM(yellowCards) |  
+-----+-----+  
| Chelsea | 7 |  
| Aston Villa | 7 |  
| Arsenal | 14 |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>   
  
mysql> SELECT team, COUNT(*) FROM players GROUP BY team;  
+-----+-----+  
| team | COUNT(*) |  
+-----+-----+  
| Chelsea | 2 |  
| Aston Villa | 1 |  
| Arsenal | 2 |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>   
  
mysql> 
```

Filtering Grouped Data

WHERE filters out rows that don't meet criteria

HAVING filters out groups that don't meet criteria

```
david.toth@cscdata: ~  
File Edit View Search Terminal Help  
mysql> select * from players;  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| lastName | firstName | birthDate | goals | assists | yellowCards | redCards | team | position |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
| Bright | Millie | 1993-01-04 | 0 | 0 | 5 | 1 | Chelsea | DF |  
| Cuthbert | Erin | 1993-05-23 | 4 | 5 | 2 | 0 | Chelsea | MF |  
| Daly | Rachel | 1991-08-21 | 12 | 3 | 7 | 1 | Aston Villa | FW |  
| Foord | Caitlin | 1994-10-15 | 5 | 7 | 2 | 0 | Arsenal | FW,MF |  
| McCabe | Katie | 1995-11-17 | 1 | 6 | 12 | 2 | Arsenal | DF,FW |  
+-----+-----+-----+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> 
```

Filtering Grouped Data

Get teams and the largest number of yellow cards a player on that team has, but only if the player with the most has > 5 yellow cards

```
mysql> SELECT team, MAX(yellowCards) FROM players GROUP BY team HAVING MAX(yellowCards) >5 ORDER BY MAX(yellowCards) ASC;
+-----+-----+
| team      | MAX(yellowCards) |
+-----+-----+
| Aston Villa | 7 |
| Arsenal    | 12 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT team, MAX(yellowCards) FROM players GROUP BY team HAVING MAX(yellowCards) >5 ORDER BY MAX(yellowCards) DESC;
+-----+-----+
| team      | MAX(yellowCards) |
+-----+-----+
| Arsenal    | 12 |
| Aston Villa | 7 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> █
```

Sorting Filtered Data

Tack on an ORDER BY clause!

```
mysql> SELECT team, MAX(yellowCards) AS yellows FROM players GROUP BY team HAVING MAX(yellowCards) >5;
```

```
+-----+-----+
| team      | yellows |
+-----+-----+
| Aston Villa |      7 |
| Arsenal    |     12 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> SELECT team, MAX(yellowCards) AS yellows FROM players GROUP BY team HAVING MAX(yellowCards) >5 ORDER BY yellows DESC;
```

```
+-----+-----+
| team      | yellows |
+-----+-----+
| Arsenal    |     12 |
| Aston Villa |      7 |
+-----+-----+
2 rows in set (0.00 sec)
```

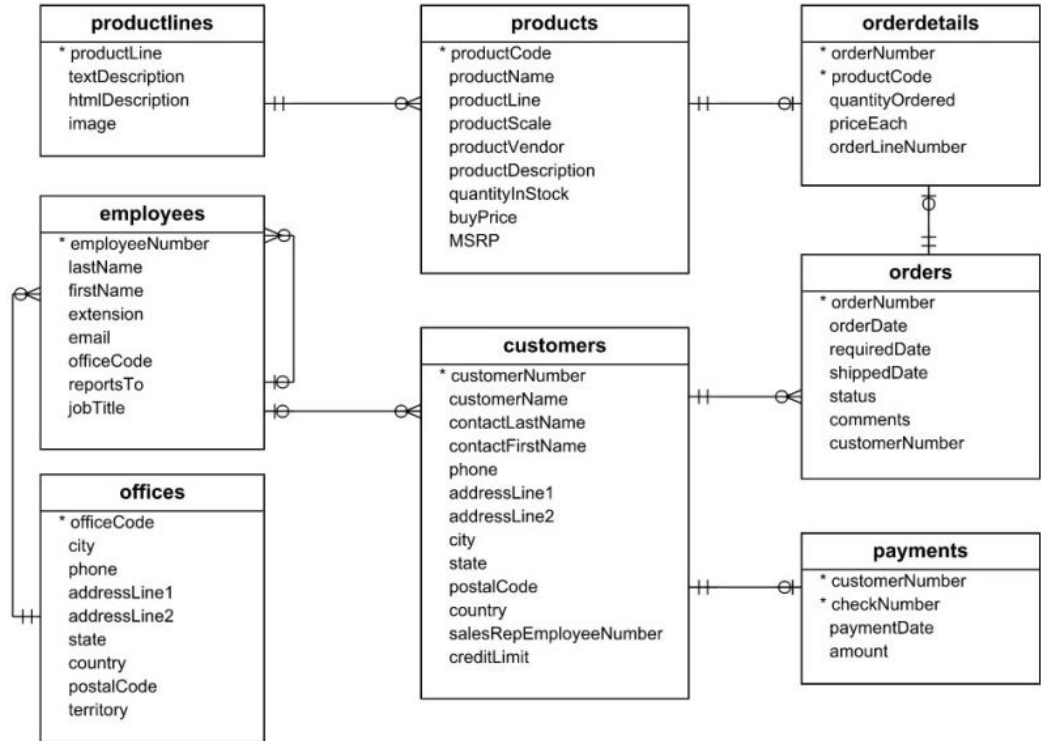
```
mysql> SELECT team, MAX(yellowCards) AS yellows FROM players GROUP BY team HAVING MAX(yellowCards) >5 ORDER BY yellows ASC;
```

```
+-----+-----+
| team      | yellows |
+-----+-----+
| Aston Villa |      7 |
| Arsenal    |     12 |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> █
```

ER Diagrams

Diagrams show DB
tables (& more)



JOIN

Lots of JOINS, but we're just doing INNER JOIN (default if you just use JOIN)

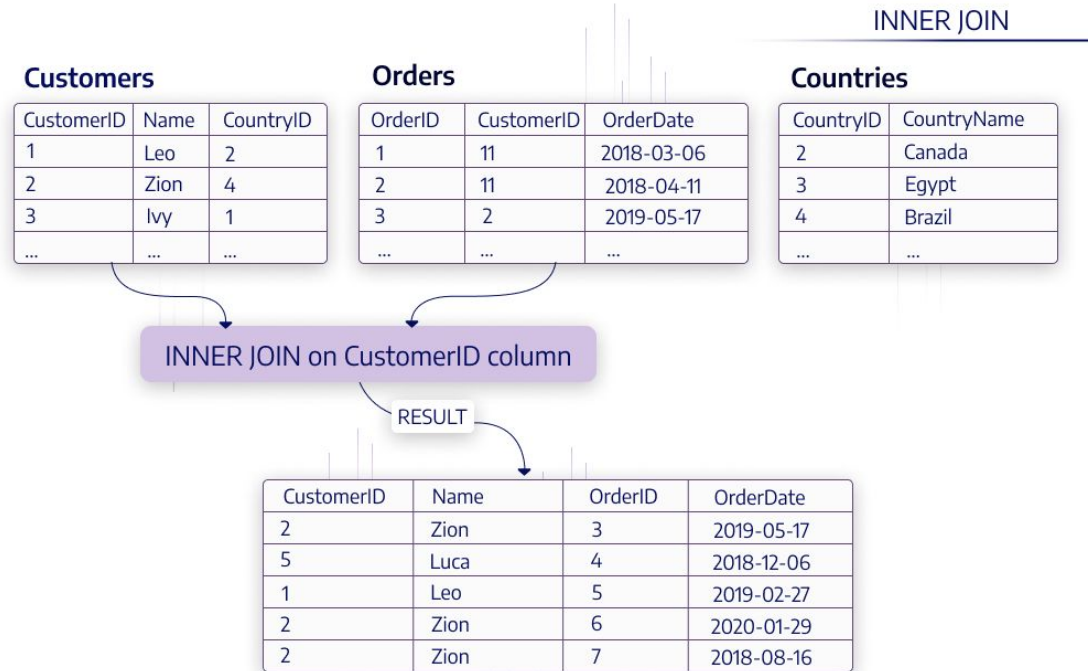
Cross product of rows - restrict to correct things using WHERE clause

Need to avoid ambiguity with what you select - refer to columns as table.column if there's ambiguity

```
SELECT tableA.column1, tableB.column2...  
FROM tableA  
INNER JOIN tableB  
ON tableA.id_field = tableB.id_field;
```

JOIN

```
SELECT
Customers.CustomerID,
Customers.Name,
Orders.OrderID,
Orders.OrderDate from
Customers JOIN Orders
WHERE
Customers.CustomerID =
Orders.CustomerID;
```



Lab Time!

Lab is linked from Moodle.

Databases & SQL Day 4

SQL Review Day

Just reviewing the first 3 SQL labs & solutions today!

See Moodle for the solutions.

Databases & SQL Day 5

Why Do We Want to Access SQL with Python?

- Some things are a pain or impossible to do in SQL
- Sometimes we need to export our data to another program for
 - Processing (like with Excel, R, or Python & Pandas)
 - Analyzing
 - Graphing/Displaying

Accessing the Database With Python

Now let's make sure everything works

```
import MySQLdb
```

```
print("ok")
```

Simple Database Program

There's a DB called classicmodels

```
mysql> use classicmodels;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed
```

classicmodels has some tables

Including an offices table

We'll just select everything from
the offices table

```
mysql> show tables;  
+-----+  
| Tables_in_classicmodels |  
+-----+  
| customers                |  
| employees                 |  
| offices                   |  
| orderdetails              |  
| orders                    |  
| payments                  |  
| productlines              |  
| products                  |  
+-----+
```

```
mysql> describe offices;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| officeCode | varchar(10)    | NO   | PRI | NULL     |       |  
| city       | varchar(50)    | NO   |     | NULL     |       |  
| phone      | varchar(50)    | NO   |     | NULL     |       |  
| addressLine1 | varchar(50)  | NO   |     | NULL     |       |  
| addressLine2 | varchar(50)  | YES  |     | NULL     |       |  
| state      | varchar(50)    | YES  |     | NULL     |       |  
| country    | varchar(50)    | NO   |     | NULL     |       |  
| postalCode | varchar(15)    | NO   |     | NULL     |       |  
| territory  | varchar(10)    | NO   |     | NULL     |       |  
+-----+-----+-----+-----+-----+-----+
```

What We Should See

```
mysql> SELECT * FROM offices;
```

officeCode	city	phone	addressLine1	addressLine2	state	country	postalCode	territory
1	San Francisco	+1 650 219 4782	100 Market Street	Suite 300	CA	USA	94080	NA
2	Boston	+1 215 837 0825	1550 Court Place	Suite 102	MA	USA	02107	NA
3	NYC	+1 212 555 3000	523 East 53rd Street	apt. 5A	NY	USA	10022	NA
4	Paris	+33 14 723 4404	43 Rue Jouffroy D'abbans	NULL	NULL	France	75017	EMEA
5	Tokyo	+81 33 224 5000	4-1 Kioicho	NULL	Chiyoda-Ku	Japan	102-8578	Japan
6	Sydney	+61 2 9264 2451	5-11 Wentworth Avenue	Floor #2	NULL	Australia	NSW 2010	APAC
7	London	+44 20 7877 2041	25 Old Broad Street	Level 7	NULL	UK	EC2N 1HN	EMEA

```
7 rows in set (0.00 sec)
```

Simple Database Program

cscdata.centre.edu

```
import MySQLdb

print("ok")

db = MySQLdb.connect("██████████", "██████████", "██████████", "classicmodels")
cursor = db.cursor()
sql = "SELECT * FROM offices"
try:
    print("try to run query")
    cursor.execute(sql)
    print("query ran ok")
    db.commit()
    results = cursor.fetchall()
    for item in results:
        print(item)
except:
    print("error running query")

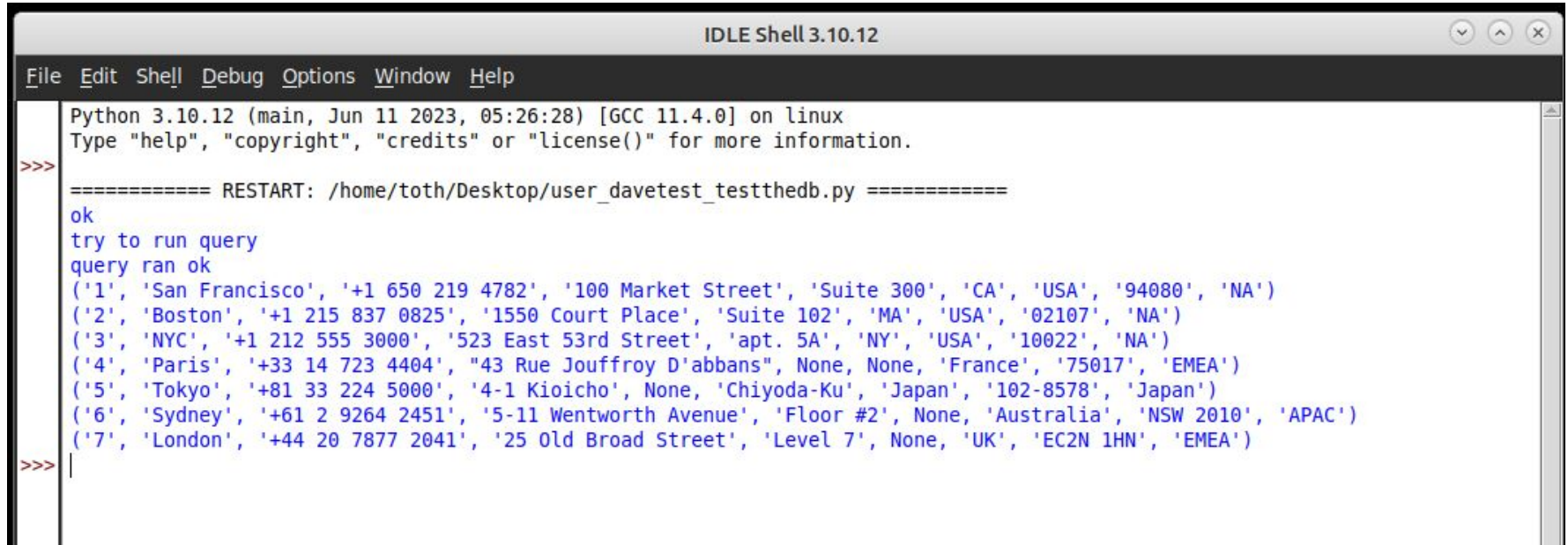
db.close()
```

your database
password

your database username

Only
needed if
changing
data

What We Do See

A screenshot of an IDLE Shell window titled "IDLE Shell 3.10.12". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area shows the following content:

```
Python 3.10.12 (main, Jun 11 2023, 05:26:28) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: /home/toth/Desktop/user_davetest_testthedb.py =====
ok
try to run query
query ran ok
('1', 'San Francisco', '+1 650 219 4782', '100 Market Street', 'Suite 300', 'CA', 'USA', '94080', 'NA')
('2', 'Boston', '+1 215 837 0825', '1550 Court Place', 'Suite 102', 'MA', 'USA', '02107', 'NA')
('3', 'NYC', '+1 212 555 3000', '523 East 53rd Street', 'apt. 5A', 'NY', 'USA', '10022', 'NA')
('4', 'Paris', '+33 14 723 4404', '43 Rue Jouffroy D'abbans', None, None, 'France', '75017', 'EMEA')
('5', 'Tokyo', '+81 33 224 5000', '4-1 Kioicho', None, 'Chiyoda-Ku', 'Japan', '102-8578', 'Japan')
('6', 'Sydney', '+61 2 9264 2451', '5-11 Wentworth Avenue', 'Floor #2', None, 'Australia', 'NSW 2010', 'APAC')
('7', 'London', '+44 20 7877 2041', '25 Old Broad Street', 'Level 7', None, 'UK', 'EC2N 1HN', 'EMEA')

>>> |
```

What's Next?

Need to dissect the variable results to get what we want.

```
import MySQLdb

print("ok")

db = MySQLdb.connect("██████████", "██████████", "██████████", "classicmodels")
cursor = db.cursor()
sql = "SELECT * FROM offices"
try:
    print("try to run query")
    cursor.execute(sql)
    print("query ran ok")
    db.commit()
    results = cursor.fetchall()
    for item in results:
        print(item)
except:
    print("error running query")

db.close()
```

your username

your password



What Does It Do? What Do We Get?

`cursor.fetchall()` fetches all the rows of a query result. It returns all the rows as a list of tuples. An empty list is returned if there is no record to fetch.

`cursor.fetchmany(size)` returns the number of rows specified by `size` argument. When called repeatedly, this method fetches the next set of rows of a query result and returns a list of tuples. If no more rows are available, it returns an empty list.

`cursor.fetchone()` method returns a single record or None if no more rows are available.

<https://pynative.com/python-cursor-fetchall-fetchmany-fetchone-to-read-rows-from-table/>

What's a Tuple?

We got a list of tuples.

Tuple:

- Stores multiple items in one variable
- Ordered
- Immutable
- Allows duplicates
- Accessed like lists (with `indexing` or `for item in the_tuple`)
- Can do `len(some_tuple)`

What's Our Tuple Look Like?

A 2D list!

```
for row in range(len(results)):
    numCol = len(row)
    for col in range(len(numCol)):
        print(results[row][col], end = ",")
    print("")
```

Speaking of Graphing...

See plotting slides...

Lab Time!

Lab linked from Moodle

Databases & SQL Day 6

Modules & Data Ninjas!

Modules

See modules slides

Databases & SQL Day 7

Plotting Lab Review & Continuation

Databases & SQL Day 8

Creating Databases, Creating & Deleting Tables,
Inserting, Updating, & Deleting Data

Creating Databases

-- Create a database with the name dbname

```
CREATE DATABASE dbname;
```

Data Types

INT: -2,147,483,648 to 2,147,483,647

DECIMAL: a float that allows you to specify num of permissible digits

DECIMAL(5,2) allows -999.99 to 999.99

DOUBLE: a double precision floating point num

DATE: YYYY-MM-DD

DATETIME: YYYY-MM-DD HH:MM:SS

YEAR: 1901-2155 as YY or YYYY

CHAR: fixed num of characters, padding to fill out

VARCHAR: up to set num of characters, no padding

Creating & Deleting Tables

Simplified creation - leaving out lots of options

CREATE TABLE or CREATE TABLE IF NOT EXISTS

CREATE TABLE table_name(name type, name type, ...);

Example:

```
CREATE TABLE players(name varchar(32), goals int);
```

DROP TABLE table_name;

Inserting Data

INSERT INTO table_name VALUES(value,
value...);

Make sure to enclose text values in quotes

INSERT INTO pet VALUES("Blitz", "Dave",
"dog", "m", DATE("2022-04-28"), NULL);

```
mysql> select * from pet;
+-----+-----+-----+-----+-----+-----+
| name   | owner | species | sex | birth      | death |
+-----+-----+-----+-----+-----+-----+
| Puffball | Diane | hamster | f   | 1999-03-30 | NULL  |
| Fluffy   | Harold | cat     | f   | 1993-02-04 | NULL  |
| Claws    | Gwen  | cat     | m   | 1994-03-17 | NULL  |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> INSERT INTO pet VALUES("Blitz", "Dave", "dog", "m", DATE("2022-04-28"), NULL);
Query OK, 1 row affected (0.00 sec)

mysql> select * from pet;
+-----+-----+-----+-----+-----+-----+
| name   | owner | species | sex | birth      | death |
+-----+-----+-----+-----+-----+-----+
| Puffball | Diane | hamster | f   | 1999-03-30 | NULL  |
| Fluffy   | Harold | cat     | f   | 1993-02-04 | NULL  |
| Claws    | Gwen  | cat     | m   | 1994-03-17 | NULL  |
| Blitz    | Dave  | dog     | m   | 2022-04-28 | NULL  |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Updating Data

UPDATE table SET column-name
= value WHERE column-name =
value;

UPDATE pet SET name = "Magic"
WHERE name = "Blitz";

**BE CAREFUL OF THE WHERE
CLAUSE!!! There's no undo!**

```
mysql> select * from pet;
+-----+-----+-----+-----+-----+-----+
| name   | owner | species | sex | birth      | death |
+-----+-----+-----+-----+-----+-----+
| Puffball | Diane | hamster | f   | 1999-03-30 | NULL  |
| Fluffy  | Harold | cat     | f   | 1993-02-04 | NULL  |
| Claws   | Gwen  | cat     | m   | 1994-03-17 | NULL  |
| Blitz   | Dave  | dog     | m   | 2022-04-28 | NULL  |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> UPDATE pet SET name = "Magic" WHERE name = "Blitz";
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from pet;
+-----+-----+-----+-----+-----+-----+
| name   | owner | species | sex | birth      | death |
+-----+-----+-----+-----+-----+-----+
| Puffball | Diane | hamster | f   | 1999-03-30 | NULL  |
| Fluffy  | Harold | cat     | f   | 1993-02-04 | NULL  |
| Claws   | Gwen  | cat     | m   | 1994-03-17 | NULL  |
| Magic   | Dave  | dog     | m   | 2022-04-28 | NULL  |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Deleting Data

Remove all rows

```
DELETE FROM table;
```

Remove some rows

```
DELETE FROM table WHERE column  
= value
```

Example:

```
DELETE FROM pet WHERE name =  
"Puffball";
```

Again, NO undo!

```
mysql> select * from pet;  
+-----+-----+-----+-----+-----+-----+  
| name   | owner | species | sex | birth      | death |  
+-----+-----+-----+-----+-----+-----+  
| Puffball | Diane | hamster | f   | 1999-03-30 | NULL  |  
| Fluffy  | Harold | cat     | f   | 1993-02-04 | NULL  |  
| Claws   | Gwen  | cat     | m   | 1994-03-17 | NULL  |  
| Magic   | Dave  | dog     | m   | 2022-04-28 | NULL  |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.01 sec)  
  
mysql> DELETE FROM pet WHERE name = "Puffball";  
Query OK, 1 row affected (0.01 sec)  
  
mysql> select * from pet;  
+-----+-----+-----+-----+-----+-----+  
| name   | owner | species | sex | birth      | death |  
+-----+-----+-----+-----+-----+-----+  
| Fluffy  | Harold | cat     | f   | 1993-02-04 | NULL  |  
| Claws   | Gwen  | cat     | m   | 1994-03-17 | NULL  |  
| Magic   | Dave  | dog     | m   | 2022-04-28 | NULL  |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Database Exam

Closed book, closed notes, etc., except on the exam, I will list the following words:

Questions will all be SQL queries

12 Questions @ 1 pt each marked right or wrong. Take number right, multiply by 8 to get 96%, and add 4 free points to get to 100%. That gives 10 minutes per question.

You'll have access to the database through putty so you can test your answers.

AS	JOIN
ASC	LIKE
AVG	LIMIT
BETWEEN	MAX
COUNT	MIN
DELETE	NOT
DESC	ORDER BY
DISTINCT	SELECT
DROP	SUM
FROM	UPDATE
GROUP BY	VALUES
HAVING	WHERE
INSERT	

Lab Time!

Lab linked from Moodle

Finished The Lab? Start The Project!